

Explainable AI for Cybersecurity: Interpreting Machine Learning Decisions in Intrusion Detection Systems

Rajat Kumar* and Sadhil Madan**

Department of Information Technology, K. J. Somaiya School of Engineering, Mumbai, India

*ORCID: 0009-0004-1688-0855

Abstract—Machine-learning intrusion detection systems can classify large volumes of network traffic, but their predictions often provide little evidence that a security analyst can examine. This study compares Random Forest and XGBoost on the official UNSW-NB15 training and test partitions for binary attack detection, then evaluates SHAP and LIME explanations across true positives, true negatives, false positives, and false negatives. At a fixed 0.5 threshold, Random Forest obtained 0.906 F1, 0.982 recall, 0.229 false-positive rate, and 0.984 ROC-AUC; XGBoost obtained 0.896 F1, 0.983 recall, 0.259 false-positive rate, and 0.984 ROC-AUC. Random Forest therefore provided the stronger precision and false-alarm trade-off, while XGBoost missed 65 fewer attacks. Global SHAP rankings shared five of their ten leading attributes but had only moderate overall rank correlation. Median top-five SHAP-LIME overlap ranged from 0.111 to 0.429, showing that local explanations often emphasized different evidence. TreeSHAP was substantially faster than LIME, while LIME fidelity was higher for XGBoost. These results support explanation as diagnostic evidence rather than proof that an alert is correct or causal.

Index Terms—cybersecurity, explainable artificial intelligence, intrusion detection system, LIME, SHAP

I. INTRODUCTION

Intrusion detection systems examine network events for signs of misuse, attack, or policy violation. Machine-learning models extend this task beyond fixed signatures by learning statistical patterns from labelled traffic. This flexibility is valuable when attack behaviour varies, but it also creates an operational problem: a model may label a flow as malicious without giving the analyst a reason that can be checked against the traffic context.

An unexplained false positive can waste investigation time or encourage alert fatigue. An unexplained false negative is more serious because harmful activity may pass without review. Accuracy alone hides this difference. A useful evaluation must therefore report recall and false-positive rate, preserve the natural test distribution, and examine why correct and incorrect predictions receive their scores.

Explainable artificial intelligence provides methods for describing a trained model's behaviour. SHAP assigns contributions to input features through an additive framework derived from Shapley values [3]. LIME fits a simple surrogate around one prediction after creating nearby samples [2]. Both techniques can help expose influential attributes, but neither proves that a prediction is correct or that an attributed feature caused an attack. Explanations can also change with reference data, perturbation rules, or random seeds.

This paper proposes a controlled comparison of Random Forest and XGBoost on UNSW-NB15. It evaluates detection and explanation under one data protocol, then separates explanation analysis by true positive, true negative, false positive, and false negative outcomes. The central question is: How effectively can Explainable AI techniques improve the interpretability of machine-learning-based intrusion detection systems while maintaining reliable intrusion-detection performance?

The study uses interpretability in a limited, testable sense. An explanation is more useful for this experiment when it identifies a compact set of understandable network attributes,

approximates the model faithfully near the selected record, remains reasonably stable under controlled reruns, and can be generated within a recorded time. These properties do not by themselves show that an analyst understands the alert or will make a better response decision. That broader claim requires a separate human evaluation.

The paper contributes a reproducible full-split benchmark, an error-stratified SHAP-LIME comparison, and a practical account of explanation agreement, fidelity, stability, and time. The implementation files retain the trained pipelines, generated figures, and machine-readable measurements so that every reported number can be checked against the same run.

II. BACKGROUND AND RELATED WORK

A. Explainability Methods

Explainability can describe an entire model or one decision. Global explanations summarise behaviour across many records, for example by ranking features using mean absolute attribution. Local explanations describe why a particular record received a score. IDS evaluation needs both levels: global summaries can reveal persistent reliance on a feature, while local explanations can show why a single alert differs from the usual pattern. A global ranking should not be used as though it were the explanation for every alert.

LIME explains a prediction by sampling around the selected record, obtaining model outputs for those samples, and fitting an interpretable local model [2]. Its model-agnostic design is useful when the same explanation procedure must be applied to different classifiers. The local approximation can nevertheless become unstable when sampling changes or when perturbations create implausible combinations of network attributes.

The choices made around LIME are part of the experiment rather than minor software settings. Kernel width controls how strongly distant samples influence the surrogate. The number of perturbed samples affects runtime and the precision of the fitted local model. Because UNSW-NB15

includes related counts, durations, rates, and protocol fields, perturbations may be mathematically valid but unlike feasible flows. The implementation preserved categorical values and interpreted LIME as a local approximation.

SHAP represents a prediction as a baseline plus feature contributions [3]. Tree-specific implementations can efficiently explain tree ensembles such as Random Forest and XGBoost. A SHAP summary can describe global model reliance, while waterfall or bar plots can explain one alert. Attributions remain conditional on the model, feature representation, background sample, and dependence assumptions. Correlated traffic features can divide or redistribute importance in ways that require cautious interpretation.

For a prediction $f(x)$, an additive explanation can be written conceptually as $f(x) = \phi_0 + \sum \phi_i$, where ϕ_0 is the reference output and ϕ_i is the contribution assigned to feature i . Positive and negative contributions are meaningful only relative to the selected class and output scale. This study used raw-output TreeSHAP values and probability-based LIME surrogates, so attribution magnitudes were not compared directly across methods.

B. Explainable Intrusion Detection

Wang et al. presented an early explainable machine-learning framework for IDS using SHAP with one-versus-all and multiclass classifiers on NSL-KDD [4]. The explanations corresponded to known attack characteristics, establishing a practical link between feature attribution and intrusion analysis. The use of an older benchmark and the limited treatment of explanation reliability leave room for evaluation on other traffic representations.

Patil et al. trained decision tree, Random Forest, SVM, and voting models on CICIDS2017 and used LIME to explain individual results [5]. Their study shows that ensemble detection and local explanation can be integrated. It concentrates on LIME and reported classifier performance, so agreement with another explainer and a systematic comparison of false positives and false negatives remain separate questions.

Wei et al. developed xNIDS to explain deep network intrusion models while accounting for input history and feature dependencies [6]. The framework evaluates fidelity, sparsity, completeness, and stability and can convert explanations into defence rules. This work demonstrates that explanation quality has measurable dimensions beyond visual appeal, although its deep-model setting differs from the tabular tree ensembles considered here.

Arreche et al. proposed XAI-IDS and evaluated seven classifiers with SHAP and LIME on CICIDS2017, NSL-KDD, and RoEduNet-SIMARGL2021 [7]. Their framework compares model behaviour, feature selection, and explanation techniques across datasets. It confirms that broad multi-model XAI comparisons already exist; a defensible contribution for the present project must therefore come from its controlled UNSW-NB15 protocol and error-stratified analysis.

Gaspar et al. applied LIME and SHAP to an MLP-based IoT intrusion solution using a generated form of the ADFA-LD system-call dataset [8]. They used perturbation analysis to test explanations and collected participant feedback. Their work supports explicit evaluation of explanations, while its system-call features and neural model limit direct transfer to network-flow tree models.

Mia et al. compared SHAP patterns for correct predictions and errors across CIC-IoT2023, NF-UQ-NIDS-v2, and HIKARI-2021 [9]. Their overlapping plots are designed to guide analysts in identifying potential false positives and false negatives. The approach gives the current study a direct methodological precedent; the planned extension compares a second explainer, retains the original test distribution, and reports explanation time and stability.

Hermosilla et al. compared SHAP and LIME for XGBoost and TabNet on UNSW-NB15, including stability and fidelity considerations in a forensic setting [10]. This is the closest prior configuration and prevents a broad novelty claim based only on the dataset and explainers. The present study instead pairs XGBoost with Random Forest and organises the comparison around all four confusion-matrix outcomes.

Mohale and Obagbuwa's systematic review found continued problems in standardisation, scalability, and real-time use of explanations in IDS [11]. Taken together, the literature shows that explanations should be evaluated as outputs with their own limitations. It also shows that explanation plots alone do not establish human understanding or operational trust.

The studies also use different meanings of explanation quality. Some judge whether a plot appears consistent with known attack characteristics. Others perturb important features, compare a surrogate with the original model, test stability, or ask participants to rate usefulness. These measures are complementary rather than interchangeable. The present study therefore uses several measurable properties and labels each one precisely. It does not collapse fidelity, stability, agreement, sparsity, runtime, and human trust into one interpretability score.

Table I. Selected literature and its relevance to the present study

Study	Method	Dataset	Main finding	Limitation or gap
Wang et al. [4]	SHAP with neural IDS	NSL-KDD	Local and global attributions align with attack characteristics	Older dataset; limited explanation validation
Patil et al. [5]	Ensemble IDS with LIME	CICIDS2017	Local explanations accompany an ensemble classifier	Single explainer; limited error-stratified analysis
Wei et al. [6]	xNIDS	Four DL-NIDS settings	Measures fidelity, sparsity, completeness, and stability	Deep sequential setting differs from tabular tree models
Arreche et al. [7]	Seven models; SHAP and LIME	CICIDS2017, NSL-KDD,	Broad model and explainer comparison	Does not centre on UNSW-NB15 error outcomes

Study	Method	Dataset	Main finding	Limitation or gap
		RoEduNet		
Gaspar et al. [8]	MLP; LIME and SHAP	Generated ADFA-LD	Perturbation and user feedback evaluate explanations	System-call data; one model family
Mia et al. [9]	SHAP error-group plots	Three traffic datasets	Diagnoses FP and FN patterns	No LIME comparison; visual similarity depends on analyst judgement
Hermosilla et al. [10]	XGBoost, TabNet; SHAP and LIME	UNSW-NB15	Compares fidelity and stability for forensic use	Different model pairing; broader outcome-stratified replication is useful
Mohale and Obagbuwa [11]	Systematic review	Twenty studies	Finds standardisation and scalability gaps	Review evidence does not replace controlled experiments

III. PROBLEM STATEMENT AND RESEARCH GAP

High-performing IDS classifiers can still be difficult to audit. A model comparison based only on accuracy cannot show whether one system misses more attacks, creates more false alarms, or relies on features that are unstable under a different explanation sample. Existing research has already applied SHAP and LIME to IDS, compared multiple models, and studied misclassifications [7]–[10]. The research gap is therefore specific rather than absolute.

The contribution is a reproducible comparison of Random Forest and XGBoost under one UNSW-NB15 protocol. Detection metrics and explanation measures are reported together. Explanations are compared across the same records where possible and stratified by confusion-matrix outcome. The study also records agreement, stability, fidelity, and processing time. This design treats the work as a scoped replication and extension, not a new XAI algorithm.

Three controls make the comparison useful. First, both classifiers receive the same predictor definitions, training records, validation logic, and final test records. Second, the original test distribution is preserved so that false-positive rate and precision describe the benchmark rather than a rebalanced sample. Third, explanations are aggregated from transformed columns to original network attributes before rankings are compared. Without these controls, differences may reflect preprocessing or presentation choices instead of the models.

The study's practical question is diagnostic. If a model flags benign traffic, do its explanations resemble typical attack detections or expose an unusual reliance on a small number of features? If a model misses an attack, which attributes pushed the score toward benign traffic, and does the competing model use those attributes differently? These questions do not promise automatic error correction. They define evidence that can guide later review of data quality, features, thresholds, and model design.

IV. RESEARCH QUESTIONS AND OBJECTIVES

A. Research Questions

RQ1: How do Random Forest and XGBoost compare in attack recall, false-positive rate, F1-score, ROC-AUC, and PR-AUC under identical data partitions?

RQ2: Which original network attributes most strongly influence each model, and how much do their global feature rankings overlap?

RQ3: How do SHAP and LIME explanations differ among true positives, true negatives, false positives, and false negatives?

RQ4: How repeatable are the explanations under controlled reruns, and what computation time do they require?

RQ5: Which diagnostic observations can be made from error explanations, and which claims about analyst trust remain untested without a user study?

B. Objectives

- Build a leakage-controlled preprocessing and training pipeline using the official UNSW-NB15 partitions.
- Compare Random Forest and XGBoost using the same test records and clearly defined positive class.
- Generate global and local explanations with network attributes presented in their original semantic groups.
- Analyse correct predictions, false alarms, and missed attacks, then measure explanation agreement, sensitivity, fidelity, and time.

V. METHODOLOGY AND IMPLEMENTATION

The implemented sequence comprised dataset acquisition, schema audit, preprocessing, training-only model selection, final model fitting, intrusion prediction, performance evaluation, SHAP and LIME explanation, and error-group comparison. The scripts save fitted pipelines, machine-readable result files, figures, case identifiers, random seeds, and software versions.

The predictor matrix contained 42 attributes. The identifier, binary label, and attack-category label were excluded; the attack category was retained only for error analysis. Protocol, service, and state were one-hot encoded with unknown-category handling, while numeric features were passed to the tree models without scaling. No test labels informed preprocessing or fitting.

Fig. 1. Implemented experimental and explanation workflow

Stage	Output
1. Audit and split	Verified schema, counts, labels, and untouched test partition
2. Preprocess	Training-fitted cleaning, encoding, and feature map
3. Train	Tuned Random Forest and XGBoost pipelines
4. Evaluate	Probabilities, confusion outcomes, and detection metrics
5. Explain	SHAP and LIME outputs for global and outcome-group samples
6. Compare	Agreement, stability, fidelity, runtime, and error interpretation

A. Dataset Description

UNSW-NB15 was generated in the Cyber Range Lab at UNSW Canberra using IXIA PerfectStorm to combine normal activity with synthetic contemporary attacks [1]. The complete collection contains 2,540,044 records and 49 attributes including the class label. Its published training and test files contain 175,341 and 82,332 records, respectively. Attack categories include Analysis, Backdoors, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms [1].

The primary task is binary classification, with attack as the positive class. The attack category is excluded from predictors and retained only for error reporting. The fixed test partition supports repeatability, while its laboratory origin and age restrict claims about modern production traffic.

The feature set describes a flow from several perspectives. Basic attributes cover protocol, service, state, duration, bytes, and packet counts. Content and time-related attributes describe connection behaviour, while additional generated features summarise related activity involving hosts, services, ports, and timing [1]. These groups create useful semantic material for explanation, but they also contain dependencies. Duration, rate, bytes, and packet counts cannot be treated as independent physical quantities when local samples are generated.

Class imbalance affects both model fitting and interpretation. The binary task contains many attacks overall while some categories remain rare. A high aggregate recall can therefore coexist with weaker recall in a category such as Fuzzers. Category errors are reported as counts and rates, with restrained interpretation for small denominators.

Table II. UNSW-NB15 study profile

Property	Implemented use
Training records	175,341; preprocessing, selection, and fitting
Test records	82,332; final evaluation only
Prediction task	Benign versus attack
Attack category	Evaluation metadata for missed-attack analysis
Feature types	Numeric and categorical network-flow attributes

B. Data Preprocessing and Feature Selection

The supplied files contained 175,341 training rows and 82,332 test rows with the expected 45 columns. The training labels comprised 119,341 attacks and 56,000 benign flows; the test labels comprised 45,332 attacks and 37,000 benign flows. The identifier, label, and attack category did not enter either predictor.

Protocol, service, and state were one-hot encoded by a transformer fitted only on training data; unseen test categories were ignored rather than causing failure. The remaining 39 numeric features were passed through unchanged because the selected tree models do not require standardisation. The same fitted transformer was embedded in both saved model pipelines.

The experiment retained all 42 predictors and the original class distribution. No resampling, test-set balancing, or test-driven feature selection was performed. This full-feature design keeps the comparison focused on the classifiers and explainers and prevents a feature-selection loop from contaminating the final evaluation.

C. Machine Learning Models

Random Forest builds an ensemble of decision trees from bootstrapped samples and random feature subsets [12]. Averaging across trees can reduce the variance of a single tree, while class weights and depth controls help manage imbalance and complexity. XGBoost adds trees sequentially to correct current errors and uses regularisation, shrinkage, and sampling to control overfitting [13]. Both models support probability estimates and tree-based SHAP, which permits a consistent explanation interface.

A fixed 20% stratified validation subset of the training file compared three configurations per model using seed 42. The selected Random Forest used 220 trees, unrestricted depth, minimum leaf size two, square-root feature sampling, and balanced bootstrap weights. The selected XGBoost model used 260 trees, depth six, learning rate 0.08, and row and column subsampling of 0.85.

Both final pipelines were refitted on all training records and evaluated at the declared 0.5 decision threshold. Continuous scores were retained for ROC-AUC and PR-AUC. No probability calibration was added, so probability magnitudes are interpreted as model scores rather than calibrated event frequencies.

D. Explainable AI Techniques

Approximate TreeSHAP produced raw-output attributions for each tree ensemble. Contributions from one-hot columns were summed back to their original attributes. Global rankings used a label-stratified sample of 1,000 test records, containing 500 benign and 500 attack flows.

LIME called each complete preprocessing-and-model pipeline. Categorical perturbations were constrained to observed protocol, service, and state values. Each explanation used 1,500 perturbations. Its local surrogate R^2 was retained as fidelity evidence rather than assuming the surrogate matched the classifier.

Local comparison used 15 seeded records from each of TP, TN, FP, and FN for each model. The top-five feature-set Jaccard score measured SHAP-LIME overlap. LIME stability used three seeds on five records per outcome. TreeSHAP repeatability was checked under identical model and input settings; because the implemented tree-path procedure is deterministic, this is a repeatability check rather than a background-sensitivity experiment.

E. Experimental Setup

The implementation used Python 3.12.14, scikit-learn 1.9.1, XGBoost 3.4.1, SHAP 0.52.0, LIME 0.2.0.1, and seed 42 on Windows 11. The official test file remained outside

model selection. The 0.5 threshold was fixed before test scoring.

All reported samples were selected by code from stored predictions. The balanced explanation sample supports comparisons among TP, TN, FP, and FN but does not estimate their frequency in natural traffic. Runtime was measured for the implemented batch SHAP call and individual LIME calls on the same machine; the resulting values are comparative rather than hardware-independent benchmarks.

F. Evaluation Metrics

Accuracy, precision, recall, F1, false-positive rate, ROC-AUC, and PR-AUC measure detection. For binary labels,

precision = $TP/(TP+FP)$, recall = $TP/(TP+FN)$, F1 is their harmonic mean, and false-positive rate = $FP/(FP+TN)$. Confusion counts and attack-category misses provide the denominators hidden by aggregate rates.

Explanation comparison used top-five Jaccard overlap, directional agreement on shared features, LIME surrogate R^2 , seeded LIME stability, deterministic SHAP repeatability, and median time per record. These measures answer different questions; agreement or stability does not prove that an explanation is true.

VI. RESULTS AND ANALYSIS

Table III. Test performance on 82,332 records at threshold 0.5

Model	Accuracy	Prec.	Recall	F1	FPR	ROC-AUC	PR-AUC
Random Forest	0.8872	0.8403	0.9818	0.9055	0.2287	0.9840	0.9880
XGBoost	0.8745	0.8232	0.9833	0.8962	0.2587	0.9840	0.9884

Random Forest produced TN=28,538, FP=8,462, FN=823, and TP=44,509. XGBoost produced TN=27,428, FP=9,572, FN=758, and TP=44,574. XGBoost detected 65 additional attacks, but Random Forest avoided 1,110 false alarms and therefore achieved higher precision, F1, and accuracy. Their ROC-AUC values were nearly identical; PR-AUC slightly favoured XGBoost.

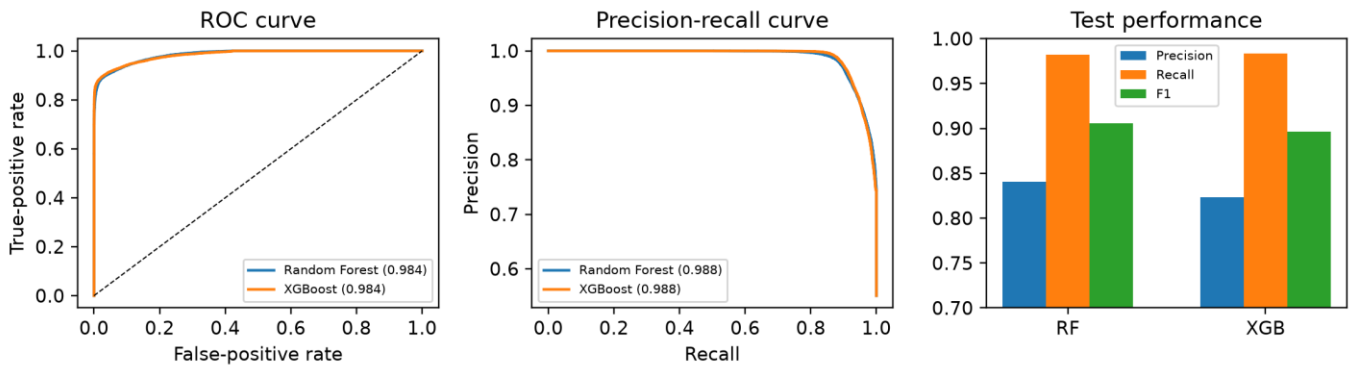


Fig. 2. ROC, precision-recall, and fixed-threshold performance on the official test split

A. Explainability Analysis

In the 1,000-record global sample, sttl ranked first for both models. Random Forest next emphasized ct_state_ttl, sload, state, and ct_srv_dst; XGBoost emphasized proto, service, dbytes, and response_body_len. Five attributes appeared in both top tens: sttl, service, dbytes, dttl, and ct_srv_dst. The top-ten Jaccard score was 0.333 and the full-ranking Spearman correlation was 0.483, indicating related but materially different reliance patterns.

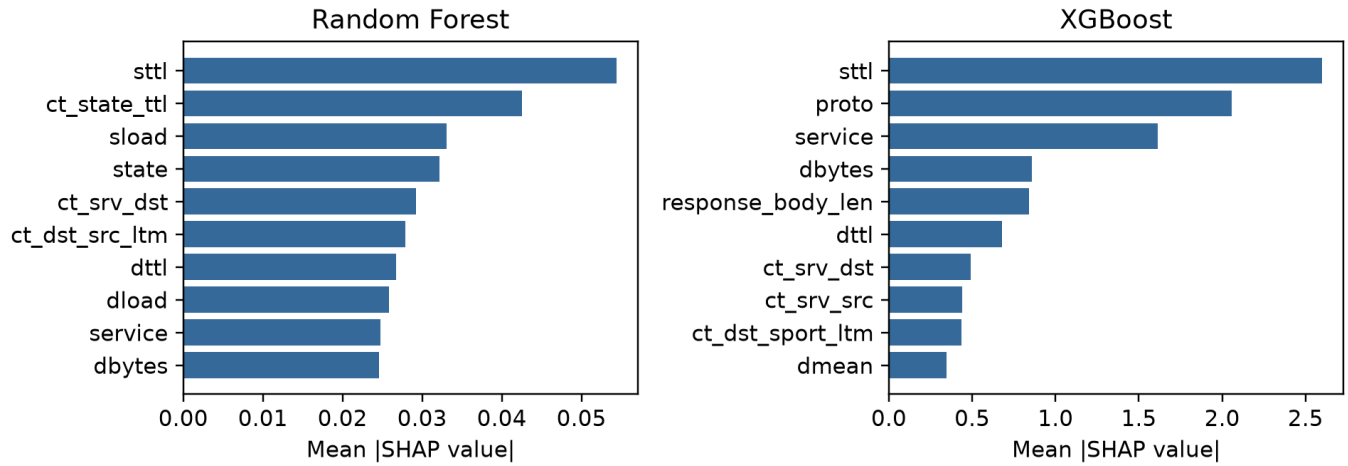


Fig. 3. Global mean absolute approximate TreeSHAP values on 1,000 stratified test records

Table IV. Outcome-stratified local explanation measurements (median values)

Model	Outcome	n	Top-5 Jacc.	LIME R ²	LIME stab.	SHAP ms	LIME ms
RF	TP	15	0.250	0.637	0.667	3.99	105.44
RF	TN	15	0.429	0.632	0.667	3.99	98.13
RF	FP	15	0.111	0.699	0.667	3.99	99.78
RF	FN	15	0.111	0.722	0.667	3.99	101.76
XGB	TP	15	0.250	0.801	0.429	1.16	61.24
XGB	TN	15	0.250	0.787	0.429	1.16	65.80
XGB	FP	15	0.111	0.776	0.429	1.16	55.32
XGB	FN	15	0.250	0.779	0.429	1.16	56.63

Top-five agreement was modest and fell to 0.111 for false positives under both models and for Random Forest false negatives. When SHAP and LIME did share a leading feature, median directional agreement was 1.0 for TP and TN, but it fell to 0.5 for FN in both models. XGBoost's LIME surrogates had higher median fidelity (0.776–0.801) than Random Forest's (0.632–0.722). LIME stability was 0.667 for Random Forest and 0.429 for XGBoost. Identical TreeSHAP calls were deterministic, but this does not test sensitivity to a different attribution convention or background distribution.

B. False Positive and False Negative Analysis

The error distribution exposes the operational trade-off. Random Forest misclassified 22.87% of benign test flows as attacks, compared with 25.87% for XGBoost. Most missed attacks were Fuzzers: 759 of 6,062 for Random Forest (12.52%) and 636 of 6,062 for XGBoost (10.49%). Random Forest had 56 missed Exploits and only eight missed attacks across the remaining reported categories; XGBoost had 63 missed Exploits, 21 DoS, 19 Analysis, six Backdoors, seven Generic, four Shellcode, and two Reconnaissance cases. No Worms were missed by either model in this run.

The low SHAP-LIME overlap for error cases means that a single local explanation should not be treated as a settled account of an error. SHAP describes the fitted tree ensemble on its raw-output scale, while LIME approximates the local probability surface created by its perturbations. Their disagreement is therefore evidence about method dependence. Feature attributions remain associations with model output and do not establish that a traffic attribute caused an attack.

VII. DISCUSSION

The results operate on two levels. Detection metrics show that both classifiers recover more than 98% of attacks, but at the cost of substantial false-positive rates. Random Forest gives the better fixed-threshold balance; XGBoost gives the highest recall by a small margin. Explanation metrics show that the account presented to an analyst changes materially with the explainer, especially for errors.

Post-hoc explanation does not change a fixed model's predictions. Adding SHAP or LIME therefore cannot directly improve accuracy, recall, or false-positive rate. Explanations may support later model redesign, feature review, or threshold

adjustment, but each change requires validation on data that did not inform the change. Once test cases guide redesign, those cases are no longer an untouched final evaluation set.

Claims about trust and practical usability require evidence from security analysts. The present benchmark can demonstrate feature visibility, explanation consistency, and processing cost. It cannot demonstrate faster triage, better analyst decisions, or greater trust without a user study. This boundary is important because readable plots may still omit dependencies or reflect artefacts in the data.

Operational preference determines the model choice. Random Forest reduced false alarms by 1,110 relative to XGBoost, which matters when benign volume is high.

XGBoost detected 65 additional attacks and missed fewer Fuzzers, which may matter when missed activity carries the dominant cost. Neither fixed threshold is deployment-ready without site-specific calibration.

Approximate TreeSHAP required about 1.16–3.99 ms per sampled record, whereas LIME required about 55–105 ms. This supports batch SHAP for broad review and selective LIME use when a second local perspective is valuable. The low overlap between methods should be displayed as uncertainty, not hidden by choosing the more persuasive explanation.

VIII. LIMITATIONS

UNSW-NB15 contains laboratory-generated traffic and may not represent present production networks, encrypted applications, or current attack behaviour. Its predefined split supports repeatability but does not test temporal generalisation or cross-network transfer. Binary labels merge distinct attacks, and rare categories may yield too few errors for stable subgroup analysis.

SHAP and LIME explain model behaviour rather than network causality. Correlated features, one-hot aggregation, approximate TreeSHAP, and LIME sampling can alter attribution. The outcome groups contain only 15 records each for explanation comparison, and the global sample contains 1,000 records. SHAP repeatability was tested under identical calls rather than alternative backgrounds. No confidence intervals or analyst study were performed, so human usefulness remains untested.

IX. FUTURE SCOPE

After the binary benchmark is complete, the protocol can be extended to multiclass attack recognition, temporal validation, and cross-dataset testing on CICIDS2017 or a newer operational collection. Counterfactual explanations could complement feature attribution by showing feasible changes associated with a different decision. A user study with security analysts could measure decision accuracy, review time, confidence calibration, and the effect of explanation uncertainty. Runtime profiling could also test selective explanation of high-risk alerts rather than every flow.

X. CONCLUSION

This study completed a reproducible Random Forest and XGBoost comparison on the official UNSW-NB15 partitions. Random Forest achieved the stronger fixed-threshold balance with 0.906 F1 and a 0.229 false-positive rate, while XGBoost achieved slightly higher recall at 0.983. SHAP exposed overlapping global reliance on sttl, service, dbytes, dttl, and ct_srv_dst, yet the models' complete rankings were only moderately correlated. Local SHAP-LIME overlap was low, especially for false positives, and LIME was markedly slower. Explainability therefore adds useful diagnostic visibility, but the method dependence and benchmark false-alarm rates

require explanations to be presented as review evidence rather than guarantees of correctness or trust.

APPENDIX A RESEARCH PAPER COMPLETION CHECKLIST

The completed paper and its retained experiment artefacts satisfy the following research and reporting checks.

- The title, abstract, keywords, introduction, background, problem statement, research gap, research questions, objectives, methodology, dataset, implementation tools, models, explanation methods, experimental setup, measured results, discussion, limitations, future scope, conclusion, and references are included.
- The work remains directly connected to AI Applications in Cyber Security through machine-learning intrusion detection, explanation of automated decisions, model reliability, and analyst-facing error diagnosis.
- The study uses one public network intrusion benchmark, two tree-ensemble classifiers, and two post-hoc explanation methods under a common evaluation protocol.
- The literature review covers eight IDS-focused publications and the references include the source dataset, LIME, SHAP, Random Forest, and XGBoost papers. Bibliographic details must receive a final reference-manager check before submission.
- Detection evaluation includes confusion counts, accuracy, precision, recall, F1, false-positive rate, ROC-AUC, PR-AUC, and attack-category recall. Explanation evaluation includes agreement, directional agreement, fidelity, stability, and processing time.
- The analysis covers true positives, true negatives, false positives, and false negatives. Case selection, explanation settings, model files, figures, and machine-readable results are stored for reproduction.
- All provisional fields have been replaced by measurements from the official test split. The abstract, results, discussion, limitations, and conclusion match the stored outputs.
- ROC, precision-recall, fixed-threshold performance, global SHAP rankings, explanation fidelity, stability, agreement, and timing are reported. The final Word document was rendered and visually checked after insertion.
- Citation order, figure and table numbering, page layout, and consistency between stored results and reported values were checked. Any institutional similarity percentage must still be produced by the institution's chosen checker.

REFERENCES

- [1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.
- [2] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
- [3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 4765–4774.
- [4] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," IEEE Access, vol. 8, pp. 73127–73141, 2020, doi: 10.1109/ACCESS.2020.2988359.

- [5] S. Patil et al., “Explainable artificial intelligence for intrusion detection system,” *Electronics*, vol. 11, no. 19, Art. no. 3079, 2022, doi: 10.3390/electronics11193079.
- [6] F. Wei, H. Li, Z. Zhao, and H. Hu, “xNIDS: Explaining deep learning-based network intrusion detection systems for active intrusion responses,” in *32nd USENIX Security Symposium*, 2023, pp. 4337–4354.
- [7] O. Arreche, T. Guntur, and M. Abdallah, “XAI-IDS: Toward proposing an explainable artificial intelligence framework for enhancing network intrusion detection systems,” *Applied Sciences*, vol. 14, no. 10, Art. no. 4170, 2024, doi: 10.3390/app14104170.
- [8] D. Gaspar, P. Silva, and C. Silva, “Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron,” *IEEE Access*, vol. 12, pp. 30164–30175, 2024, doi: 10.1109/ACCESS.2024.3368377.
- [9] M. Mia, M. M. A. Pritom, T. Islam, and K. Hasan, “Visually analyze SHAP plots to diagnose misclassifications in ML-based intrusion detection,” in *2024 IEEE International Conference on Data Mining Workshops*, 2024, pp. 632–641, doi: 10.1109/ICDMW65004.2024.00088.
- [10] P. Hermosilla, S. Berrios, and H. Allende-Cid, “Explainable AI for forensic analysis: A comparative study of SHAP and LIME in intrusion detection models,” *Applied Sciences*, vol. 15, no. 13, Art. no. 7329, 2025, doi: 10.3390/app15137329.
- [11] V. Z. Mohale and I. C. Obagbuwa, “A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity,” *Frontiers in Artificial Intelligence*, vol. 8, Art. no. 1526221, 2025, doi: 10.3389/frai.2025.1526221.
- [12] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [13] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.